Human Verification: `Containers.Directed`

Directed Containers and the Comonad They Induce

Machine-generated from `lean/Directed.lean` for review by Neil Ghani

July 2026

Abstract

This document presents every definition and theorem *statement* from the Lean 4 module `Containers.Directed` in three parallel forms: the original Lean, a readable Haskell rendering, and precise mathematical English. Proofs are omitted — they are machine-checked by Lean. The purpose is to let a human verifier confirm that the formalisation captures the intended mathematics.

Contents

1 Overview **1**

2 The extensionality lemma **2**
2.1 Theorem: `Ext.ext_eq` 2

3 Directed containers **2**
3.1 Structure: `DirectedContainer` 2

4 The comonad on `Ext C` **3**
4.1 Definition: `DirectedContainer.counit` 3
4.2 Definition: `DirectedContainer.comult` 4
4.3 Theorem: `DirectedContainer.left_counit` 4
4.4 Theorem: `DirectedContainer.right_counit` 4
4.5 Theorem: `DirectedContainer.coassoc` 5

5 Translation conventions **5**

1 Overview

This file develops *directed containers* (Ahman–Chapman–Uustalu, *When is a container a comonad?*) and shows that the extension $\mathrm{Ext}\, C$ of a directed container C carries a canonical comonad structure.

A directed container augments a container $\langle S, P\rangle$ with:

- a *root* $o(s) \in P(s)$ for each shape s;
- a *sub-shape* operation $s \downarrow p \in S$ for each position $p \in P(s)$;
- a *shift* $p \oplus q \in P(s)$ combining $p \in P(s)$ with $q \in P(s \downarrow p)$;
- five laws D1–D5 making each $P(s)$ a "monoid over the base".

The comonad data on $D := \operatorname{Ext} C$ are:

$$\varepsilon\langle s, v\rangle = v(o(s)), \qquad \delta\langle s, v\rangle = \langle s, p \mapsto \langle s \downarrow p, q \mapsto v(p \oplus q)\rangle\rangle.$$

The three comonad laws follow from D1–D5. Laws D2 and D5 carry explicit dependent transports (along D1 and D4 respectively) — this is the genuinely dependent content of the development.

2 The extensionality lemma

2.1 Theorem: `Ext.ext_eq`

Theorem 2.1 (Extensionality for Ext across a shape equality)**.** Let $s_1, s_2 : S$ with $\mathtt{hs} : s_1 = s_2$, and let $v_1 : P(s_1) \to X$, $v_2 : P(s_2) \to X$. If the labellings agree after transporting positions along `hs`,

$$\forall\, q : P(s_1), \quad v_1(q) = v_2(\mathtt{hs} \triangleright q),$$

then $\langle s_1, v_1\rangle = \langle s_2, v_2\rangle$ as elements of $\operatorname{Ext} C\, X$.

Lean.

```
theorem Ext.ext_eq {C : Container} {X : Type} {s1 s2 : C.Shape}
    (hs : s1 = s2) {v1 : C.Pos s1 → X} {v2 : C.Pos s2 → X}
    (hv : ∀ q : C.Pos s1, v1 q = v2 (hs ▸ q)) :
    (⟨s1, v1⟩ : Ext C X) = ⟨s2, v2⟩
```

Haskell.

```
-- In Haskell, Ext s p x = (s, p → x) and we have no shape equality
-- to transport along.  The analogue is simply:
-- if s1 == s2 and forall q, v1 q == v2 q, then (s1, v1) == (s2, v2).
```

Fidelity note. The expression $\mathtt{hs} \triangleright q$ is Lean's *transport*: it coerces $q : P(s_1)$ to type $P(s_2)$ along the equality $s_1 = s_2$. This is the central dependent-type operation in the entire file. It appears whenever positions in different fibres $P(s)$ must be compared. There is no Haskell analogue; in Haskell all position types are the same (a single type parameter `p`), so transports are invisible.

3 Directed containers

3.1 Structure: `DirectedContainer`

Definition 3.1 (Directed container)**.** A **directed container** extends a container $\langle S, P\rangle$ with:

- root : $(s : S) \to P(s)$ — the root position of each shape;
- sub : $(s : S) \to P(s) \to S$ — the sub-shape $s \downarrow p$ beneath position p;
- shift : $(s : S) \to (p : P(s)) \to P(s \downarrow p) \to P(s)$ — the shift $p \oplus q$;

satisfying the five laws:

D1. $s \downarrow o(s) = s$ (sub-shape under the root is the shape itself);

D2. $\forall s, q : P(s \downarrow o(s)),\ o(s) \oplus q = \mathrm{D1}(s) \triangleright q$ (shifting by the root is transport along D1);

D3. $\forall s, p : P(s),\ p \oplus o(s \downarrow p) = p$ (shift by sub-root is identity);

D4. $\forall s, p, q,\ s \downarrow (p \oplus q) = (s \downarrow p) \downarrow q$ (sub-shapes nest along shifts);

D5. $\forall s, p, q, q',\ (p \oplus q) \oplus (\mathrm{D4}^{-1} \triangleright q') = p \oplus (q \oplus q')$ (shift is associative, with transport along D4).

Lean.

```
structure DirectedContainer extends Container where
  root  : (s : Shape) → Pos s
  sub   : (s : Shape) → Pos s → Shape
  shift : (s : Shape) → (p : Pos s) → Pos (sub s p) → Pos s
  d1 : ∀ s, sub s (root s) = s
  d4 : ∀ (s : Shape) (p : Pos s) (q : Pos (sub s p)),
    sub s (shift s p q) = sub (sub s p) q
  d2 : ∀ (s : Shape) (q : Pos (sub s (root s))),
    shift s (root s) q = d1 s |→ q
  d3 : ∀ (s : Shape) (p : Pos s),
    shift s p (root (sub s p)) = p
  d5 : ∀ (s : Shape) (p : Pos s) (q : Pos (sub s p))
              (q' : Pos (sub (sub s p) q)),
    shift s (shift s p q) ((d4 s p q).symm |→ q')
      = shift s p (shift (sub s p) q q')
```

Haskell.

```
-- A directed container approximated in Haskell
-- (no dependent types; s and p are type parameters).
class (Container s p) ⇒ DirectedContainer s p where
  root  :: s → p
  sub   :: s → p → s
  shift :: s → p → p → p   -- shift s p q : p, not Pos (sub s p)
  -- Laws (as QuickCheck properties):
  -- D1: sub s (root s) == s
  -- D2: shift s (root s) q == q   (transport invisible in Haskell)
  -- D3: shift s p (root (sub s p)) == p
  -- D4: sub s (shift s p q) == sub (sub s p) q
  -- D5: shift s (shift s p q) q' == shift s p (shift (sub s p) q q')
```

Fidelity note. In Lean, the shift has the precise dependent type $\mathrm{shift}(s, p, -) : P(s \downarrow p) \to P(s)$, and laws D2 and D5 carry transport coercions (along D1 and D4 respectively) that coerce positions between fibres. In the Haskell rendering, all positions are the same type `p`, so these transports are invisible — the laws D2 and D5 still hold but without the coercions. The mathematical content (the monoid-over-the-base structure on $P(s)$) is preserved; the dependent bookkeeping is not.

4 The comonad on `Ext C`

4.1 Definition: `DirectedContainer.counit`

Definition 4.1 (Counit)**.** The **counit** $\varepsilon : \mathrm{Ext}\, C \Rightarrow \mathrm{Id}$ at X is:

$$\varepsilon\langle s, v\rangle \;=\; v(o(s)).$$

It evaluates a labelled container at the root position of its shape.

Lean.

```
def DirectedContainer.counit : Ext D.toContainer X → X :=
  fun p ⇒ p.2 (D.root p.1)
```

Haskell.

```
counit :: DirectedContainer s p ⇒ (s, p → x) → x
counit (s, v) = v (root s)

-- As a comonad extract:
instance DirectedContainer s p ⇒ Comonad (Ext s p) where
```

```
  extract (Ext s v) = v (root s)
```

4.2 Definition: `DirectedContainer.comult`

Definition 4.2 (Comultiplication)**.** The **comultiplication** $\delta : \operatorname{Ext} C \Rightarrow \operatorname{Ext} C \circ \operatorname{Ext} C$ at X is:

$$\delta\langle s, v\rangle \;=\; \Big\langle s,\; p \mapsto \big\langle s \downarrow p,\; q \mapsto v(p \oplus q)\big\rangle\Big\rangle.$$

Each position p yields a new labelled container of sub-shape $s \downarrow p$, whose labelling evaluates v at the shift $p \oplus q$.

Lean.

```
def DirectedContainer.comult :
    Ext D.toContainer X → Ext D.toContainer (Ext D.toContainer X) :=
  fun p ⇒ ⟨p.1, fun q ⇒ ⟨D.sub p.1 q, fun r ⇒ p.2 (D.shift p.1 q r)
     ⟩⟩
```

Haskell.

```
comult :: DirectedContainer s p ⇒ (s, p → x) → (s, p → (s, p → x)
   )
comult (s, v) = (s, \q → (sub s q, \r → v (shift s q r)))

-- As a comonad duplicate:
  duplicate (Ext s v) = Ext s (\q → Ext (sub s q) (\r → v (shift s q
      r)))
```

4.3 Theorem: `DirectedContainer.left_counit`

Theorem 4.3 (Left counit law)**.**

$$\varepsilon \circ \delta \;=\; \mathrm{id}_{\operatorname{Ext} C\, X}.$$

The shape equation uses D1; the value equation uses D2 (with a transport).

Lean.

```
theorem DirectedContainer.left_counit :
    D.counit  ∘  D.comult
      = (id : Ext D.toContainer X → Ext D.toContainer X)
```

Haskell.

```
-- Comonad law (left unit):
-- extract . duplicate = id
--
-- Proof: (extract . duplicate) (s, v)
--   = extract (s, \q → (sub s q, \r → v (shift s q r)))
--   = let (sub s (root s), \r → v (shift s (root s) r))
--       = (s, \r → v r)     -- by D1 and D2
--       = (s, v)
```

4.4 Theorem: `DirectedContainer.right_counit`

Theorem 4.4 (Right counit law)**.**

$$(\operatorname{Ext} C\, \varepsilon) \circ \delta \;=\; \mathrm{id}_{\operatorname{Ext} C\, X}.$$

The shapes are unchanged; the value equation uses D3.

Lean.

```
theorem DirectedContainer.right_counit :
    Ext.map D.counit  ∘  D.comult
      = (id : Ext D.toContainer X → Ext D.toContainer X)
```

Haskell.

```
-- Comonad law (right unit):
-- fmap extract . duplicate = id
--
-- Proof: (fmap extract . duplicate) (s, v)
--   = fmap extract (s, \q → (sub s q, \r → v (shift s q r)))
--   = (s, \q → extract (sub s q, \r → v (shift s q r)))
--   = (s, \q → v (shift s q (root (sub s q))))
--   = (s, \q → v q)     -- by D3
--   = (s, v)
```

4.5 Theorem: `DirectedContainer.coassoc`

Theorem 4.5 (Coassociativity)**.**

$$(\mathrm{Ext}\, C\, \delta) \circ \delta \;=\; \delta \circ \delta \;:\; \mathrm{Ext}\, C\, X \to \mathrm{Ext}\, C\, (\mathrm{Ext}\, C\, (\mathrm{Ext}\, C\, X)).$$

The inner shapes agree by D4; the deepest values agree by D5, transported along D4.

Lean.

```
theorem DirectedContainer.coassoc :
    Ext.map D.comult  ∘  D.comult
      = (D.comult  ∘  D.comult :
          Ext D.toContainer X
            → Ext D.toContainer (Ext D.toContainer (Ext D.toContainer
                X)))
```

Haskell.

```
-- Comonad law (coassociativity):
-- fmap duplicate . duplicate = duplicate . duplicate
--
-- Proof: both sides send (s, v) to
--   (s, \p → (sub s p, \q → (sub (sub s p) q, \r → v (shift s p (
    shift (sub s p) q r)))))
-- using D4 (sub s (shift s p q) = sub (sub s p) q) and
--       D5 (shift s (shift s p q) r = shift s p (shift (sub s p) q r))
    .
-- In Haskell these transports are invisible; in Lean they require
    ext_eq.
```

Fidelity note. The coassociativity proof in Lean requires two applications of `Ext.ext_eq` to deal with the shape equality D4 and the transport in D5. Specifically: the left-hand side has inner shape $s \downarrow (p \oplus q)$, while the right-hand side has inner shape $(s \downarrow p) \downarrow q$; these are equal by D4 but not definitionally. D5 then equates the value functions after transporting the position q' along D4. In the Haskell rendering (where all positions are a single type), these transports vanish and the law holds by simple equational reasoning on shift.

5 Translation conventions

1. **Transport (▷).** The Lean notation `h ▷ q` transports q along equality h. This appears in D2, D5, and the proof of `Ext.ext_eq`. It is the most important dependent-type feature of this file and has no Haskell counterpart.

2. **Sub-shape types.** In Lean, $q : P(s \downarrow p)$ — the type of q depends on both s and p. In Haskell we approximate this by a single position type `p`, losing the fibre structure.

3. **Comonad laws.** The three comonad laws are stated as equalities of functions $\mathrm{Ext}\, C\, X \to -$. In Haskell, these are the standard comonad laws `extract . duplicate = id`, `fmap extract . duplicate = id`, and `fmap duplicate . duplicate = duplicate . duplicate`.

4. **Universe level.** The Lean development is entirely at **Type** (a single universe) with no Mathlib dependency.